

## **Unit 06: Big Data Analysis Using Python Libraries**

### **6.1 Introduction to Big Data and PySpark**

#### **Big Data – Definition**

Big Data refers to extremely large datasets that cannot be processed using traditional database systems.

#### **Characteristics of Big Data (5 Vs)**

1. **Volume** – Large amount of data
2. **Velocity** – Fast data generation
3. **Variety** – Different data types (text, image, video, etc.)
4. **Veracity** – Data accuracy and reliability
5. **Value** – Useful insights from data

#### **Big Data Processing Framework**

A Big Data Processing Framework is a software system designed to store, process, and analyze very large datasets efficiently using distributed computing. These frameworks divide large tasks into smaller parts and process them across multiple machines (cluster computing).

They are designed to handle the challenges of big data such as high volume, velocity, and variety.

#### **PySpark – Definition**

PySpark is the Python programming interface of **Apache Spark**, an open-source distributed computing framework designed for big data processing.

PySpark allows developers and data analysts to write Spark applications using Python to process and analyze very large datasets across a cluster of computers efficiently.

#### **Role of PySpark in Big Data Analysis**

- Distributed data processing
- Fast in-memory computation
- Handles structured and unstructured data
- Supports machine learning

- SQL-based data querying

## 6.2 Installing and Configuring PySpark

### Requirements

- Python (3.x)
- Java (JDK 8 or above)
- pip package manager

### Installation Command

```
pip install pyspark
```

### Creating Spark Session

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("App").getOrCreate()
```

**SparkSession** is the entry point for working with Spark.

## 6.3 PySpark DataFrames

### Definition

A DataFrame in PySpark is a distributed, immutable collection of data organized into named columns, similar to a table in a relational database or a spreadsheet. It is built on top of **Apache Spark** and is designed for processing large-scale structured and semi-structured data efficiently across a cluster of machines.

Unlike traditional tables stored in a single system, a PySpark DataFrame distributes data across multiple nodes, enabling parallel processing and high performance.

### Creating DataFrame

```
data = [("Ram", 20), ("Sita", 22)]  
df = spark.createDataFrame(data, ["Name", "Age"])  
df.show()
```

### Important Operations

- `select()` – Select columns

- `filter()` – Filter rows
- `groupBy()` – Group data
- `count()` – Count records
- `describe()` – Summary statistics

### Types of Operations

- **Transformations** – Lazy operations (select, filter)
- **Actions** – Execute operations (show, count)

## 6.4 Big Data Analysis with PySpark

### Common Analysis Tasks

- Data filtering
- Aggregation (sum, avg, max, min)
- Handling missing values
- Sorting and grouping

### Example (Aggregation)

```
from pyspark.sql.functions import sum

df.groupBy("Age").sum().show()
```

### Advantages of PySpark for Big Data

- High speed (in-memory processing)
- Scalable (cluster computing)
- Fault tolerant

## 6.5 Using SQL (DBMS Tools) for Querying Data

### Spark SQL – Definition

Spark SQL is a module of Apache Spark that enables users to process structured and semi-structured data using standard SQL queries. It allows SQL-based operations to be performed on DataFrames and integrates relational processing with Spark's distributed computing capabilities.

Spark SQL acts as a bridge between traditional DBMS systems and big data frameworks by allowing users to write SQL queries while leveraging Spark's parallel processing power.

### **Steps to Use SQL in PySpark**

1. Create temporary view

```
df.createOrReplaceTempView("students")
```

2. Run SQL query

```
spark.sql("SELECT * FROM students WHERE Age > 20").show()
```

### **Advantages**

- Easy for DBMS background students
- Optimized query execution
- Supports ANSI SQL

## **6.6 Data Visualization and Reporting**

### **Purpose**

Data Visualization is the graphical representation of data using charts, graphs, and dashboards to make complex information easier to understand. It helps in identifying patterns, trends, relationships, and outliers in large datasets.

In big data analysis using PySpark, visualization is usually performed after processing data with Apache Spark, and converting the results into a format suitable for plotting.

### **Popular Python Visualization Libraries**

- **Matplotlib**
- **Seaborn**
- **Plotly**

### **Basic Example**

```
import matplotlib.pyplot as plt
```

```
pdf = df.toPandas()
```

```
pdf.plot(kind="bar")  
plt.show()
```

## Reporting Methods

- Jupyter Notebook
- PDF Reports
- Dashboards

## 6.7 Concept of Spark SQL and Its Application

### Concept

Spark SQL is a component of **Apache Spark** that enables structured data processing using standard SQL queries. It allows users to query large-scale distributed datasets stored as DataFrames or tables using familiar SQL syntax.

Spark SQL combines the power of relational database systems (DBMS) with the distributed processing capability of Spark, making it suitable for big data analytics.

### Key Components

- DataFrames
- SQL Queries
- Catalyst Optimizer
- Hive Support

### Applications

- Data warehousing
- Business intelligence
- Log analysis
- ETL processing
- Real-time analytics